Tutorials and Worked Samples for MECHENG 706 Design Projects

(Codes for the tutorials can be found in Canvas Files/Sample Codes)

Contents

Arduino 2

Arduino hardware 2

Arduino IDE 5

Arduino common syntax 9

Motors and the Mobile Base 14

Servo motors 14

Mobile base in project 16

Hardware connection with Arduino 17

Sample codes 18

Distance Measuring Infrared Sensors 26

Distance measuring infrared sensor information 26

Hardware connection with Arduino 26

Sample codes 28

Gyroscope Sensor (BNO08X) 322

Gyroscope sensor information 322

Hardware connection with Arduino 33

Sample Codes – for testing the function of the gyro sensor 34

Phototransistor 35

Phototransistor information 35

Hardware connection with Arduino 37

Sample Codes 37

Wireless Module, in a separate file

Ultrasonic Sensor, refer to MECHENG 306 Lecture Notes

Arduino

Hardware connection, programming and basic functionalities (How Arduino and Shield are connected? Key input/output pins and their functions. How is Arduino programmed? Arduino programming environment.)

Arduino board Mega, https://www.arduino.cc/

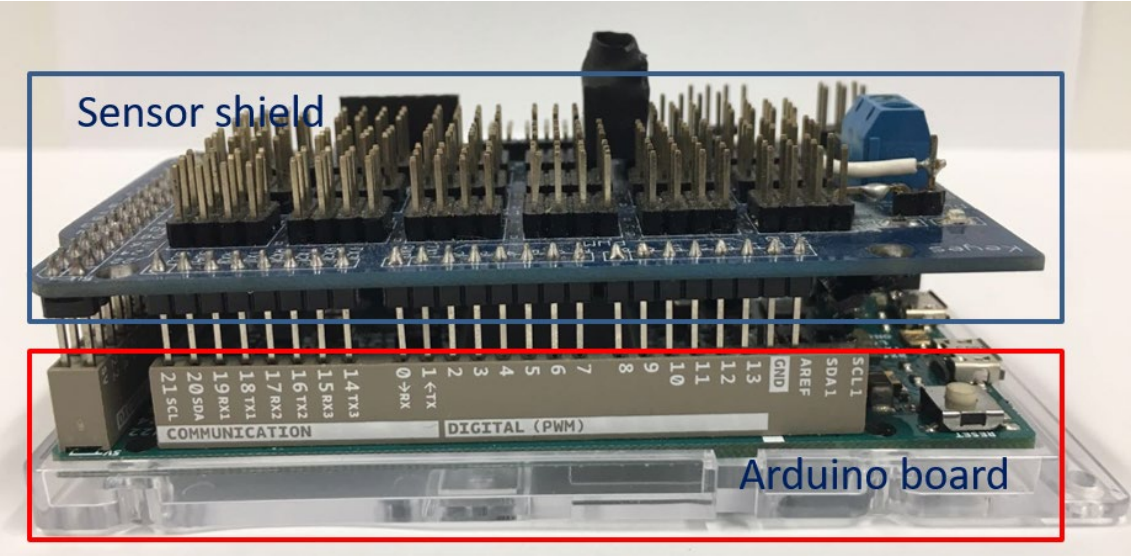


Fig.1 Arduino board and sensor shield board

Arduino hardware

The control board we use for this project is Arduino Mega, with 54 I/O pins.

ATmega microcontroller – the heart of the Arduino board.

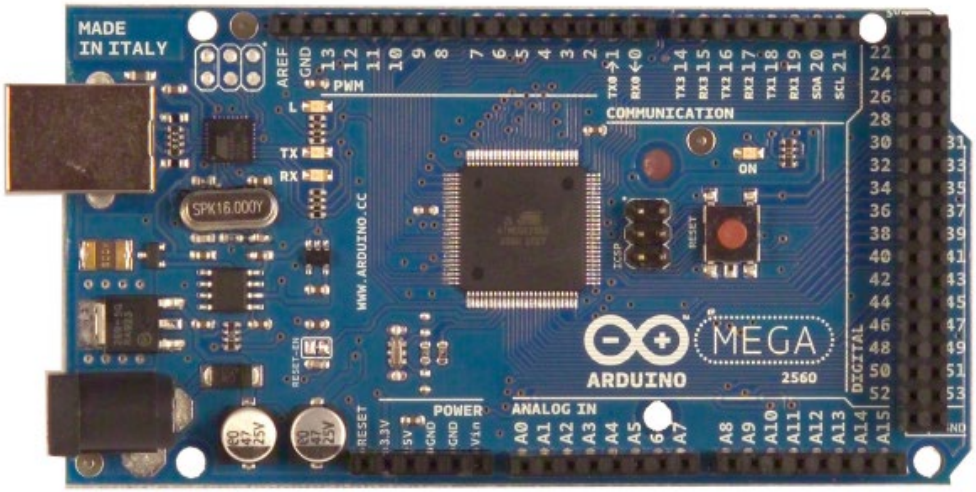


Fig.2 Arduino board

Reset Button- this will restart running the codes that were uploaded in the board.

Ground Pin (GND) and 5V pins – these pins provide +5V power to the microcontroller and other components on the board and ground to the circuits.

Analog pins (A0~A15) – these pins can read the analog input and convert it to digital using **analogRead()** function. Most of the analog pins can also be used as digital pins.

Analog in - using these pins (with initial capital A) with analogRead().

Digital pins (0-53) – these pins normally read and write two states, and they are be used for general binary input and output by using the syntax like **digitalRead (), digitalWrite ().** Every pin has an internal pull-up resistor which can be turn on and off via digitalWrite() (HIGH or LOW). However, analogWrite() also works on the digital pins (Pin 0~13) with symbols of PWM.

PWM Pin 0 ~13 – Digital pins and they output 8-bit PWM signals via **analogWrite().**

Pin13 LED - which is integrated in the board. It is very useful to test the board and debug your codes.

USB port – which is used for powering the Arduino board, uploading the codes to your Arduino and communicating with your Arduino sketch (via Serial.print() etc).

Power connector – this is another way to power your Arduino when it is not plugged into a USB port.

Power LED – indicates that the Arduino board is receiving power. Useful for debugging.

The Arduino MEGA sensor shield is to make it easy to connect to all the Digital inputs and outputs, as well as Analog inputs. It acts as an extension of the Arduino pins. It is convenient because it has designated pins for **most commonly used sensors.**

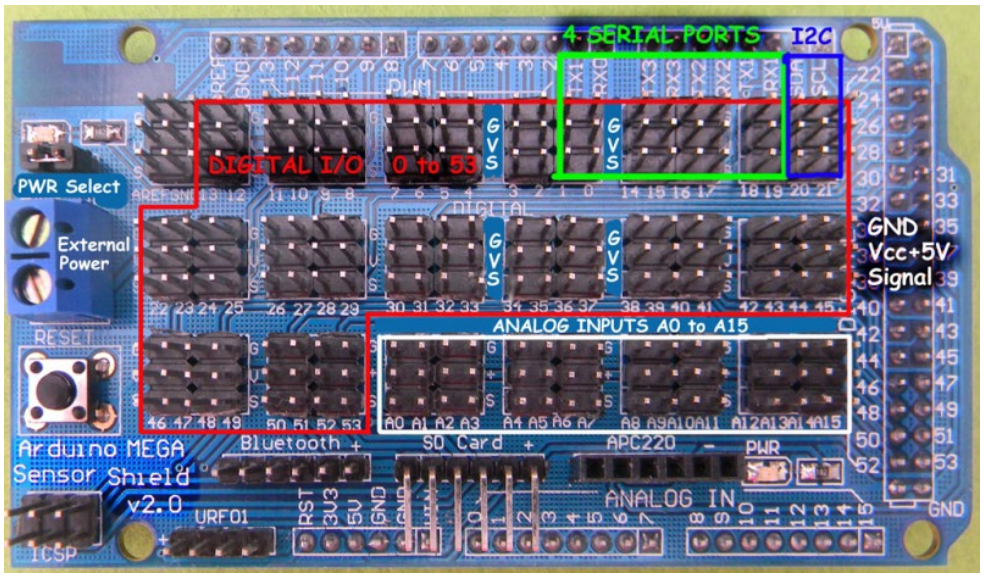


Fig.3 Arduino sensor shield board

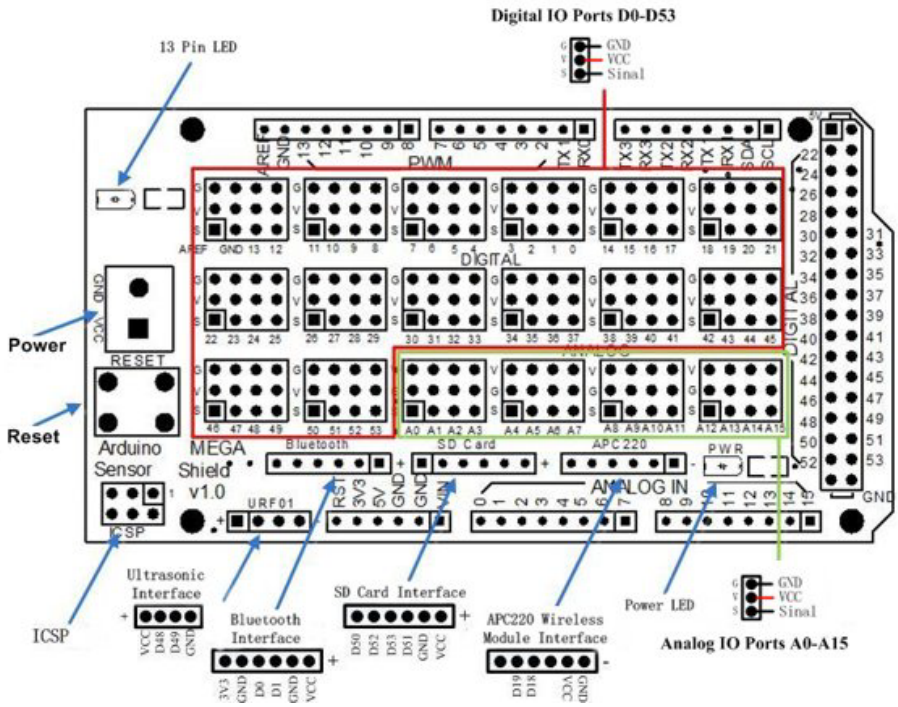


Fig.4 Arduino sensor shield board pins

It plugs directly on the top of the Arduino. More details refer to the link below,

https://www.emartee.com/product/42016/Mega%20Sensor%20Shield%20V2.0%20%20Arduino%20Compatible

Arduino IDE

Arduino IDE (integrated development environment) is a cross-platform application that is used to write and upload programmes to Arduino boards. It provides an environment for most Arduino based projects. On the top menu bar you can see the standard options, including the File (open, save etc.), Edit (copy, paste, search, etc.), Sketch (for compiling and programming), Tools (useful option for testing programme, like checking upload COM ports and board version, as well as serial monitor), and Help.

(Latest Arduino IDE, arduino-1.8.12-windows.zip, available at Canvas, Files/Sample Codes)

The middle part of the IDE is the text editing region where you can enter the programme codes. The bottom section of the IDE is to indicate the compilation and uploading status, including the information about how much memory has been used, any error was found in the programme, and other useful messages.

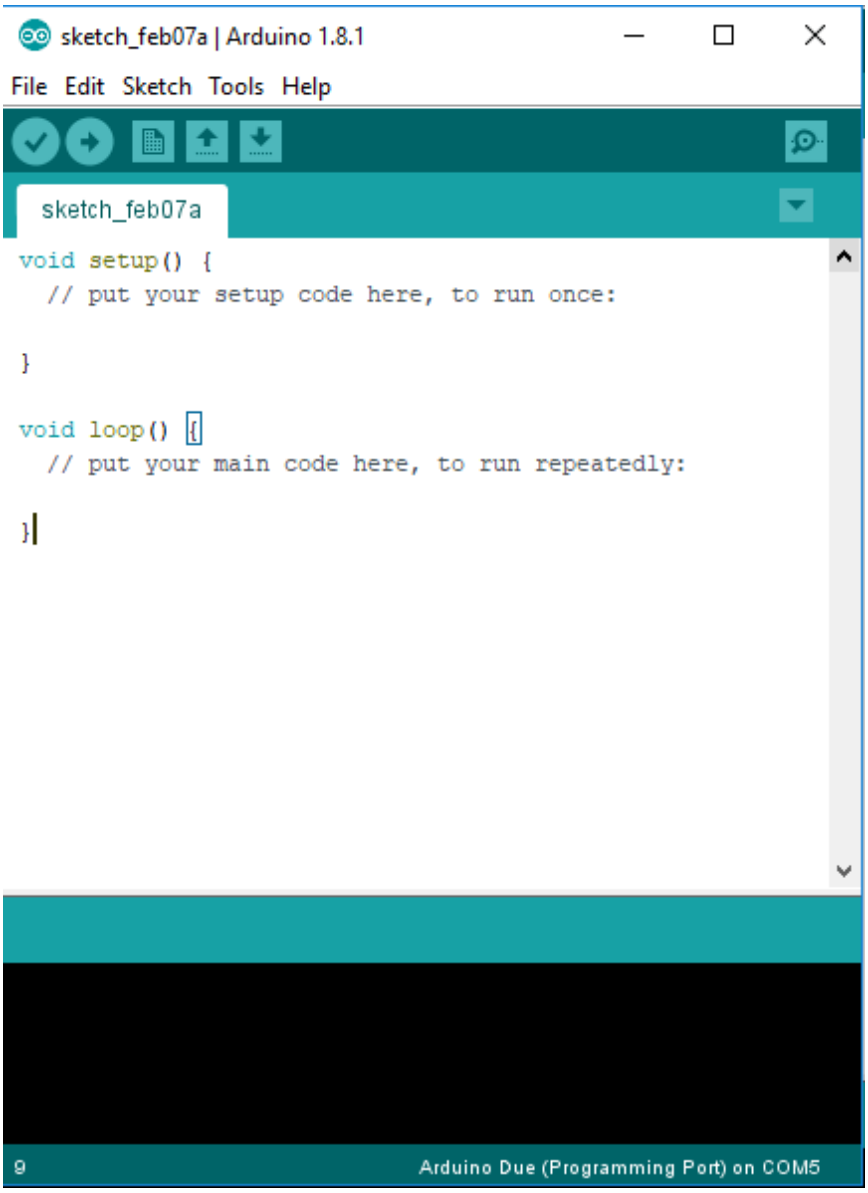


Fig.5 Arduino IED

There are a number of buttons below the menu bar.

1) the check button is used to check syntax of your codes.

2) the arrow button uploads your codes to the Arduino board to run.

3) the dotted paper will create a new file.

4) the upward arrow is to open an existing Arduino project.

5) the downward arrow is used to save current file.

6) the button in far right is a serial monitor, which is very useful to type, receive and send data between Arduino board and PC for debugging.

A simple example to upload codes

1) open the IDE, and then navigate to the LED blink example sketch. It can be found under: FILE > EXAMPLE > 01.BASICS > BLINK.

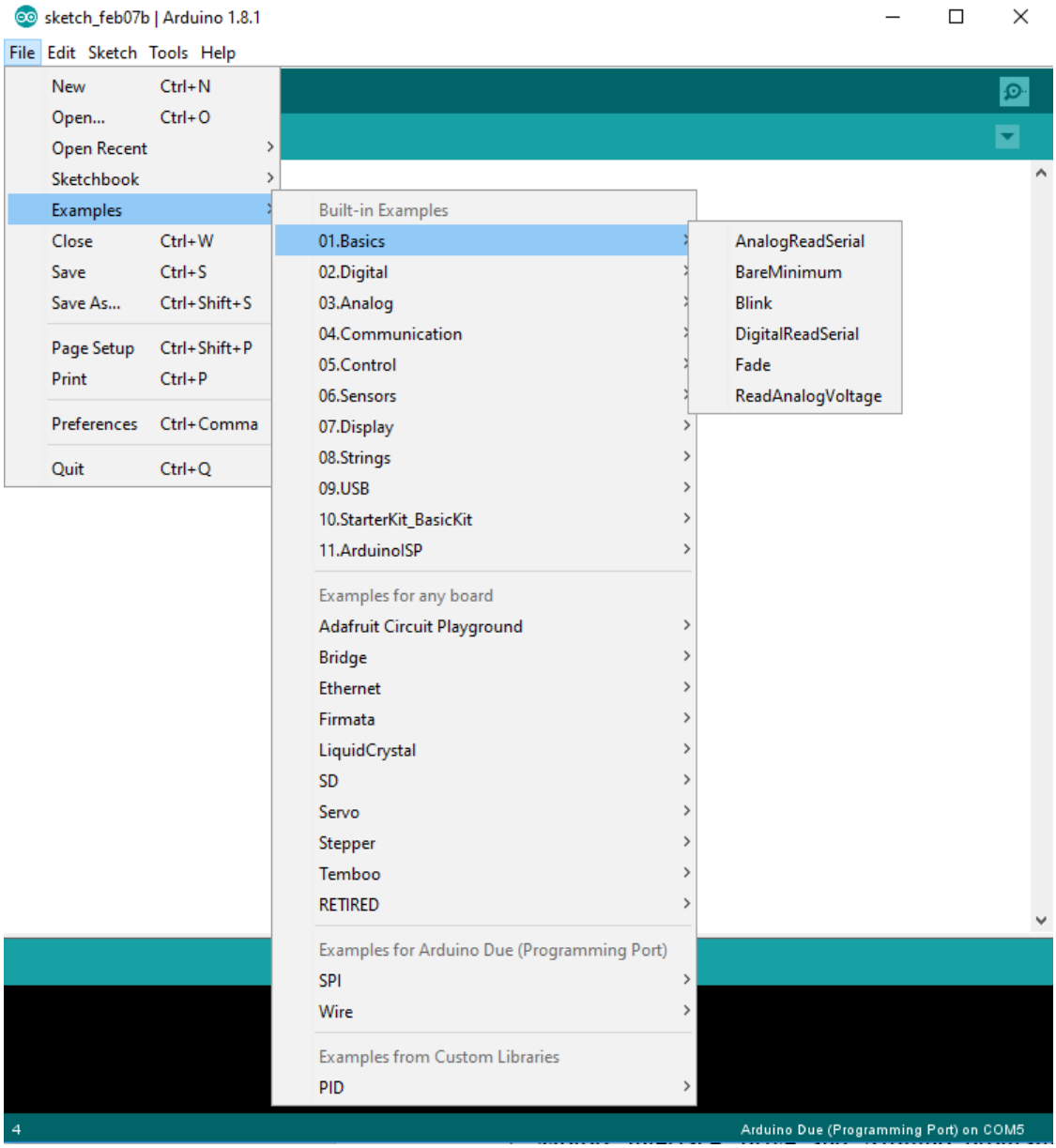


Fig.6 Arduino example code

2) A window with codes in it will be opened, and then select your board under: TOOLS > BOARDS , then find the Arduino board you are using.

3) choose the serial port your arduino is connected from the TOOLS > PORTS menu.

4) upload the sketch to you Arduino by clicking the UPLOAD toggle (an arrow symbol) in the top left corner of the window.

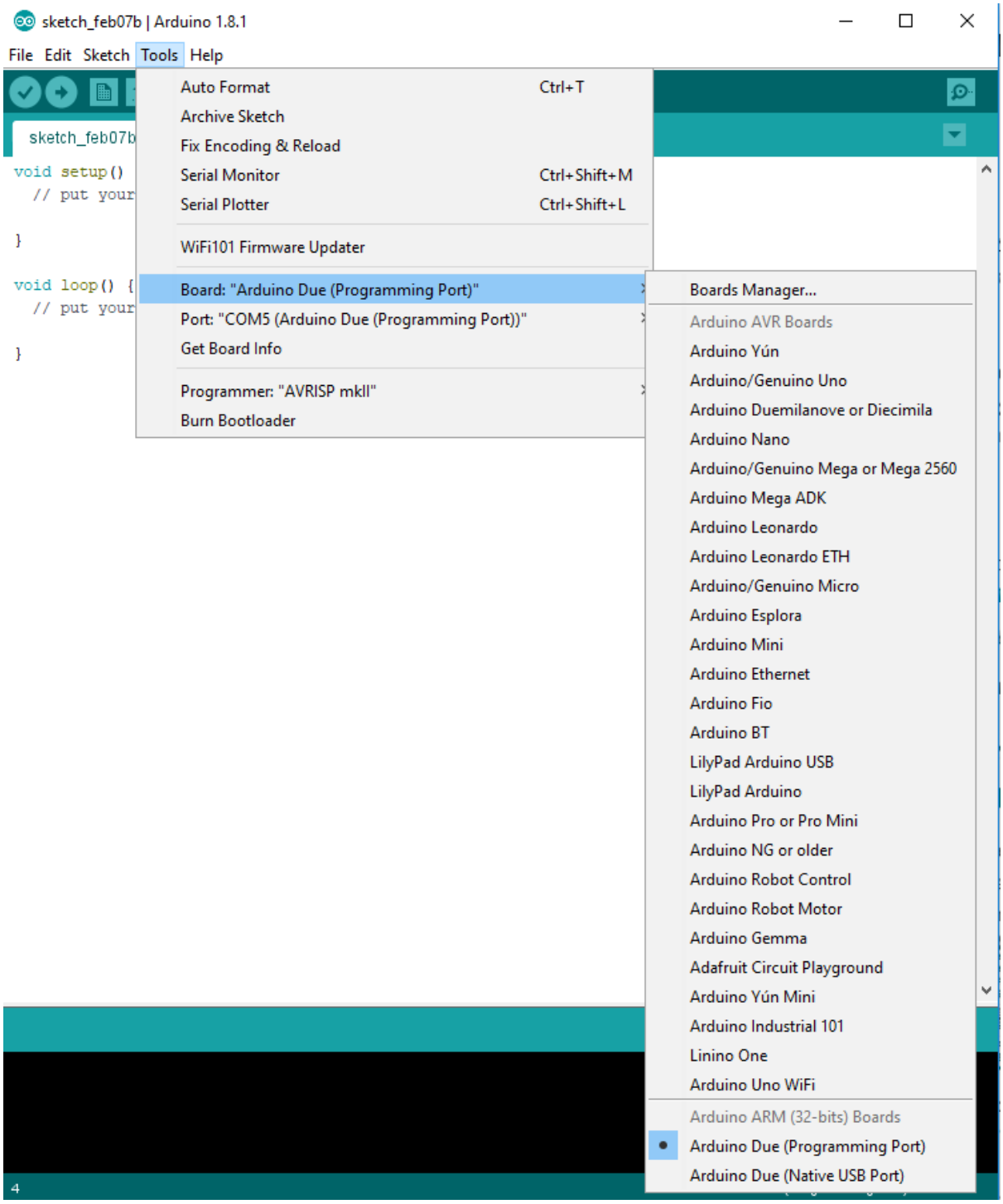


Fig.7 Arduino IDE choose board

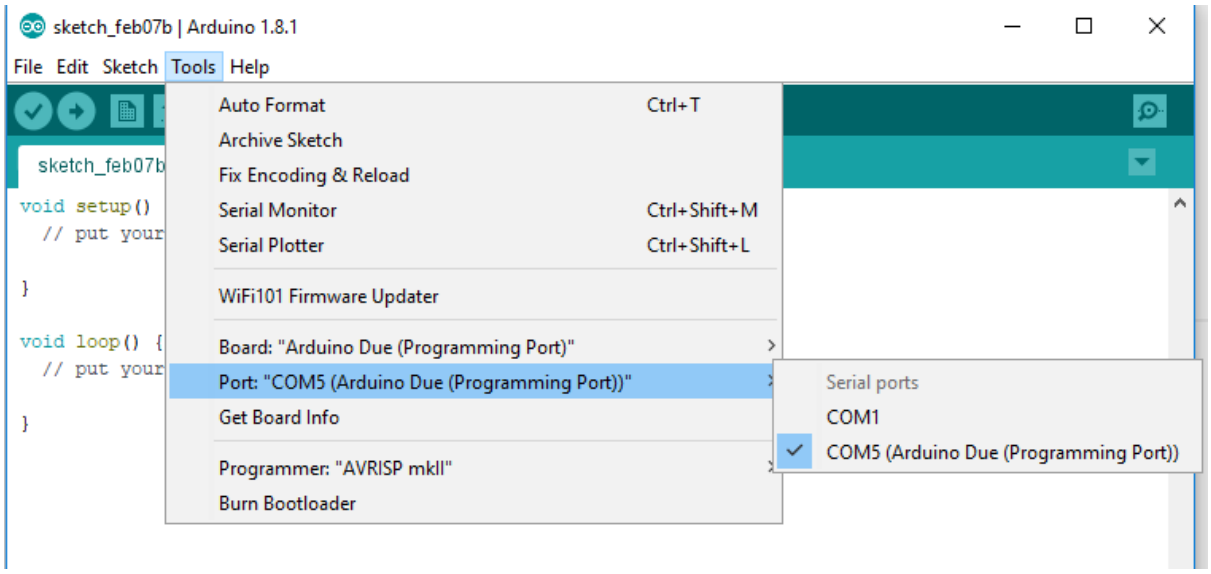


Fig.8 Arduino IDE choose upload port

5) you will see the indicating message of upload progress near the lower left corner of the IDE window, and the lights labeled TX and RX on the Arduino board be blinking. Once the upload is done, the IDE will show the message DONE UPLOADING.

6) Finally, you will see the LED with L starts blinking yellow.

Arduino common syntax

Every Arduino program has two main functions. The necessary function in an Arduino program is called *void setup ( )* which sets things up that have to be done once and then do not happen again, and *void loop ( )* is the core of your program, and contains the instructions that be repeated over and over again until the board is turn off.

```
void setup( )
{
Your codes;
}
 void loop ( )
{
Your codes;
}
```

1 in the beginning, you need to create variables and declare what types it is. You usually give it an initial value when you declare it.

2 the setup ( ) function only run once, when the Arduino is powered up. So in this section, you need to declare the *pinMode ( )* (INPUT OR OUTPUT), serial setting, etc. An example is given below,

```
void setup( )
{
pinMode(pin, OUTPUT);                    // sets the 'pin' as output
}
```

3 the loop ( ) function runs repeatedly after the setup ( ) has finished. This section is where you check for the voltage on the input, as well turn output on and off. Like *digitalWrite ( ), digitalRead ( ), analogRead ( ), analogWrite ( )*.

```
void loop ( )
{
digitalWrite(24, HIGH);    // turn 'pin 24' on
delay (100);                // pauses for 0.1 seconds
digitalWrite(28, LOW);    // turn 'pin 28' off
delay(1000);                ///pauses for 1 seconds
}
```

Functions

A function is a block of codes that has a name. It will be executed when the function is called. The void **setup** ( ) and **void loop** ( ) we mentioned before are **built-in functions**.

Custom functions can be written to complete repetitive tasks, and make the program much easier to read and modify. Functions are declared by first declaring the function type. It is the type of value to be returned by the function such ad 'int' for an integer type function. If there is no value return it would be void. After type, declare the name given to the function and in parenthesis and parameters being passed to the function.

type functionName(parameters)

{

Your codes;

}

Symbols

{} curly braces defines begin and end of a function block or a statement block.

; semicolon must be used to end a statement and separate elements of the program. Missing semicolon will lead to compile error.

/*…*/ block comments. It is an area of texts ignored by the program and used for codes description that help other to understand the program. It also is used for debugging purposes. It begins with /* and end with */.

// line comments. Single line comments begins with //. They are ignored by the program and take no memory space. It is commonly used for code description and future reminder.

Variables

All variables need to be declared before they can be used. Declaring a variable means defining its type, such as int, float, long, string, setting its name and assigning an initial value. And the value of a variable can be changed via arithmetic and assignment.

int sensorValue = 0;

The variables can be declared in the beginning of the program before *void setup*( ), inside a function, as well within a statement block such as for loops. Where the variable is declared determines the ability of certain parts of a program to make use of the variables.

A global variable can be used by every function and statement in a program. This kind of variable is declared in the beginning of a program before *void setup*( ).

A local variable is declared inside of a function or as a part of a statement. It can only be used in the scope where it is defined.

Variable type: int, byte, long, float…

Arithmetic & Operators: *, /, =, >, <…

True/false

These are Boolean constants that define logic. False is defined as 0 while true is often defined as 1. But sometimes anything except 0 is also defined as true.

High/low

You will easily see HIGH/LOW in Arduino program, they are defined pins levels as HIGH or LOW, and they are used when reading or writing to digital pins. HIGH is normally define as logic level 1, ON or 5 volts while LOW is logic level 0, OFF or 0 volts.

Input/output

These are used to define the digital pin as either INPUT or OUPUT via syntax pinMode( ).

If

If statement test whether a certain condition has been reached, and executes any statements inside the brackets if the condition is reached. If false the program will skip the statements.

if (conditions?)

{

Do somethings;

}

If …else

This statement allows more control over the flow code than *if* statement by allowing multiple tests. Each test will proceed to the next until a true test is encountered. When the true condition is encountered, its associated block of codes is run, and then the program will skip to the line following the if/else construction.

If (condition 1)

{

Do thing A

}

Else if (condition2)

{

Do things B

}

Else

{

Do things C

}

For

The *for* statement is used to repeat a block of statement enclosed in curly braces a specific time. An increment counter is often used to increment and terminate the loop.

for (initialization; condition; expression)

{

Do something;

```
}
for ( int i=0; i< 20; i++)    // declare i, test if less than 20, increment i by 1
{
digitalWrite(23,HIGH);  // turns pin23 on
delay(300);                    // pause for 0.3s
digitalWrite(35,HIGH); // turns pin 35 on
delay(300);                    // pause for 0.3s
}
```

While

While loop will loop continuously, and infinitely until the expression inside the parenthesis become false. Something much change the test variable, or the while loop will never exit.

```
while(condition)
{
Do something;
}
```

do… while

The do loop is a bottom driven loop that works in the same manner as the while loop, with the exception that the condition is tested at the end of the loop, so the do loop will always run at least once.

```
do
{
do something;
} while (some variable ?? vaule);
```

pinMode(pin, mode)

Used in setup() to configure a specific pin to behave either as an INPUT or an OUTPUT.

```
pinMode(10,OUTPUT);// pin 10 as output
```

digitalRead(5); // read from pin 5

Reads the value from a specified digital pin with the result either HIGH or LOW

digitalWrite(pin, value); // eg., pin 5, HIGH

Outputs either logic level HIGH or LOW at (turns on or off) a specified digital pin.

analogRead(pin)

Reads the values from specified analog pin with a 10-bits resolution. The resulting integer values ranges from 0 to 1023.

analogWrite(pin, value)

Writes a pseudo-analog value using hardware enabled pulse width modulation (PWM) to an output pin marked PWM. A value of 0 generates steady 0 volts output at the specified pin. A value of 255 generates steady 5 volts output at the specified pin.

delay(ms)

Pauses your program for the amount of time as specified in milliseconds, where 1000 equals 1 seconds

millis()

Returns the number of milliseconds since the Arduino board began running the current program as an unsigned long value

min(x,y)

Calculates the minimum of two numbers of any data type and returns the smaller one.

max(x,y)

Calculates the maximum of two numbers of any data type and returns the smaller one.

Serial.begin(rate)

Open serial port and sets the baud rate for serial data transmission.

```
void setup()
{
Serial.begin( );
}
```

Serial.println(data)

Print data to serial port, followed by an automatic carriage return and line feed.

Motors and the Mobile Base

Motors interface, drive and Arduino programming of Mecanum wheeled mobile base (What are the motors/specs? How 4 motors are interfaced to Arduino and how they are commanded by Arduino? Codes for the robot moving forward and turning on spot.)

Servo motors

A servo motor consists a motor and a sensor for position feedback to precisely control the position of the motor. Servo control is achieved by sending a servo a PWM (pulse-width modulation) signal. There are two types of servo motor, standard and continuous. The standard servo motor has an internal electronic device to identify the rotation angle of the motor, so the control input signal determine the motor to be positioned at various angles. However, standard servo motor has limited range of rotation, usually between 0 and 180 degree.

Continuous servo motor does not have a limit on its range of rotation, and its input determines the rotary speed and direction. **The motors in our projects are continuous servo motors which has three wires: a signal wire, a power wire and a ground wire.**

There is a library in Arduino called servo.h that includes some useful function for controlling the servo motor, and makes it easier to control servo with simple code. There are **two important syntax** ***write(dgree)*** and ***writeMicrosecond(µs)*** to control the movement of servo motor.

***write(dgree)*:** Writes a value to the servo, controlling the shaft accordingly. On a standard servo, this will set the angle of the shaft (in degrees), moving the shaft to that position. **On a continuous rotation servo, this will set the speed of the servo (with 0 being full-speed in one direction, 180 being full speed in the other, and a value near 90 being no movement).**

writeMicrosecond(µs)*:** Writes a value in microseconds (µs***) to the servo, controlling the shaft accordingly. On **a standard servo**, this will set the angle of the shaft. On standard servos a parameter value of 1000 is fully counter-clockwise, 2000 is fully clockwise, and 1500 is in the middle. **Continuous-rotation servos will respond to the *writeMicrosecond* function in an analogous manner to the *write* function, for example below,**

writeMicrosecond(1500); // the motor stops

writeMicrosecond(2000); // the clockwise full speed.

writeMicrosecond(1000); // the anti-clockwise full speed.

You can find a block of sample codes from Arduino reference page for servo motor command.

https://www.arduino.cc/en/reference/servo

```
#include <Servo.h>

Servo myservo;

void setup()

{

  myservo.attach(9); //servo signal pin

  myservo.write(90);    // set servo to mid-point

}

void loop() {}
```

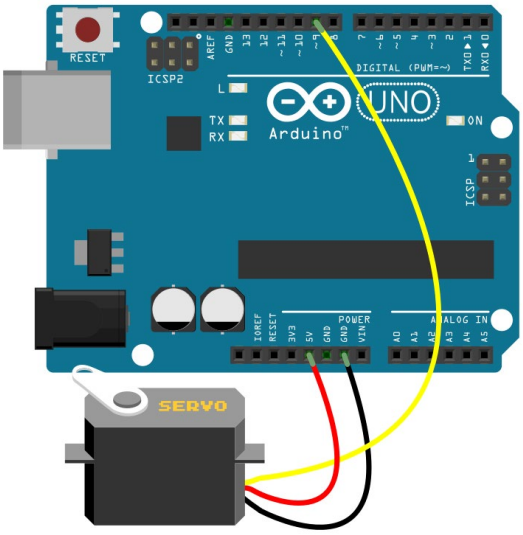


Fig.9 The hardware connection of sample code

The code instructs a servo connected to pin9 to move in its centre (90 º) position. **For a continuous servo motor, the code will stop the servo's motion.**

Most common servos accept input from 1000 µs to 2000µs, with 1500µs corresponding to the centre position, for a 0-180º servo, this would be 90 º. **In theory, the results of *write(90)* and *writeMicrosecond(*1500*)* are the same.**

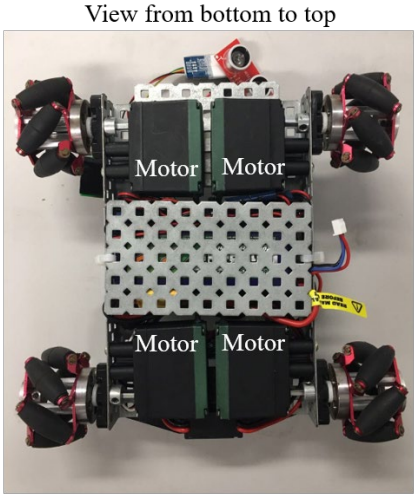
Mobile base in project

(The theory will be covered in lectures.)

The mobile base has four independent motors. The motion is determined by controlling the rotation of the four motors. Different rotation combination of the motors leads to different motion of the mobile platform. Here is a simple diagram to show the relation between motors' rotation direction and the platform moving direction.

This picture shows the mounting position of four motors, it is the view from bottom to top.

View from bottom to top



This picture is to show the mounting position of the motors.

Fig.10 Mobile platform view from bottom to top

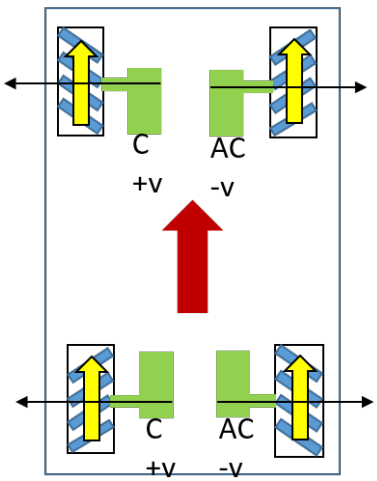
This schematic view (below) is from top to bottom, showing the motor rotation.

The yellow arrow represents the moving direction of the peak point of the wheel.
The red arrow represents the moving direction of the mobile robot.

Using the right-hand rule to determine the rotation of motors. the thumb points along the output shaft of the motor, as the black arrow points, and then

(1) if the yellow arrow points the same direction with the curl fingers, it is clockwise.
(2) If the if the yellow arrow points the opposite direction to the curl fingers, it is anticlockwise.

view from top to bottom



C :clockwise
+v : positive voltage
AC :anticlockwise
-v: negative voltage

Fig.11 Schematic diagram of mobile base motion and motors rotation

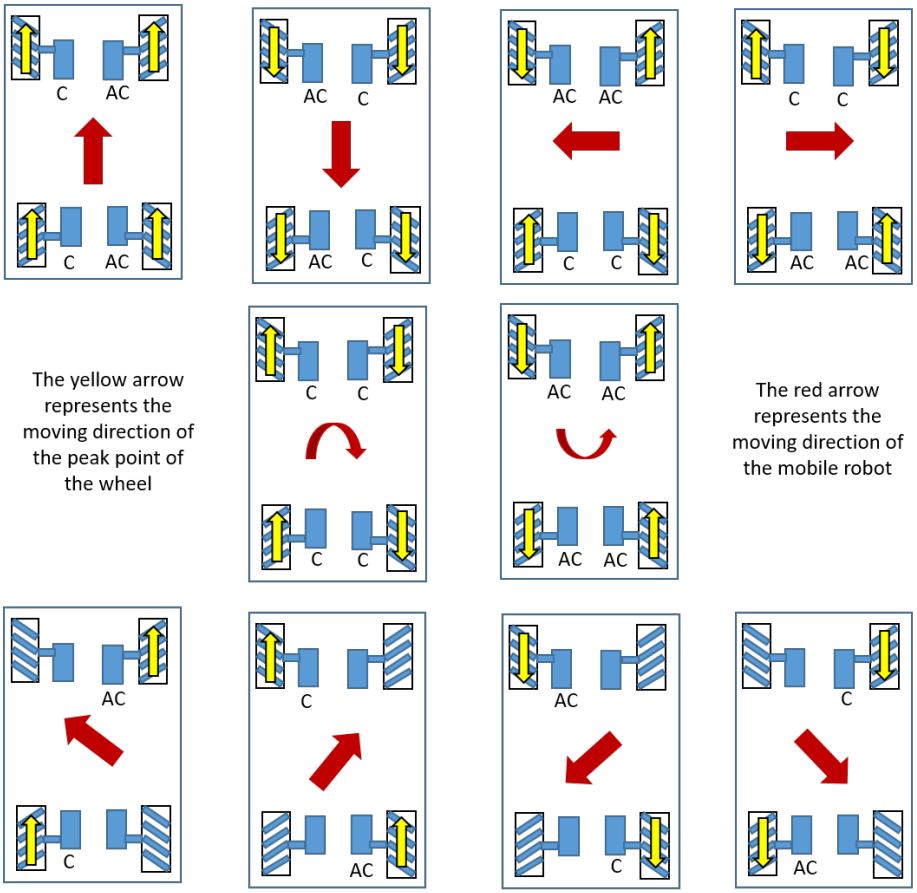
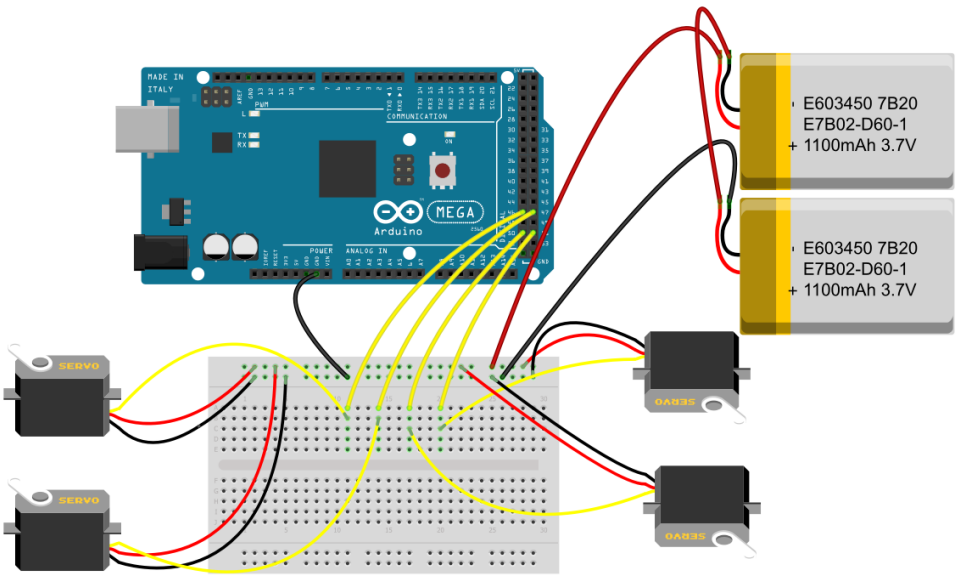


Fig.12 Schematic diagram of mobile platform motion and motors rotation combination

Hardware connection with Arduino

Here is the hardware connection of the 4 motors.

Black wires are connected to GND, red ones are power supply, and yellow wires are the signal wires.



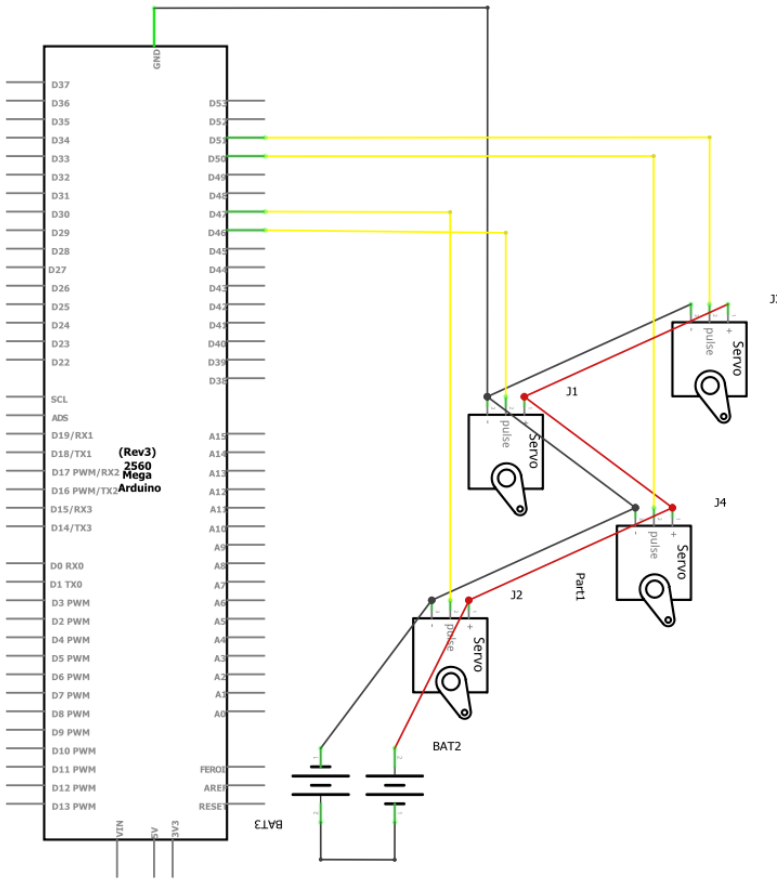


Fig.13 Four motors hardware connection with Arduino

Sample codes

– for testing basic robotic movements via serial commands

Codes (Modified from RobotBaseCodes)

```
# include <Servo.h>   // include the library of servo motor control

// define the control pin of each motor

const byte left_front = 46;

const byte left_rear = 47;

const byte right_rear = 50;

const byte right_front = 51;


// three machine states

enum STATE {

 INITIALISING,
```

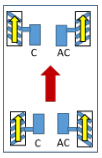
```
 RUNNING,
 STOPPED
};
// create servo objects for each motor
Servo left_front_motor;
Servo left_rear_motor;
Servo right_rear_motor;
Servo right_front_motor;

int speed_val = 100;
int speed_change; // this variable is used to change the motor speed by adding on the speed_val

void setup() {
  // put your setup code here, to run once:
Serial.begin(9600);   // start serial communication
}

void loop() {
  // put your main code here, to run repeatedly:
static STATE machine_state = INITIALISING;    // start from the sate INITIALIING
switch (machine_state)
{
  case INITIALISING:
     machine_state = initialising();
     break;
  case RUNNING:
     machine_state = running();
     break;
  case STOPPED:
     machine_state = stopped();
     break;
};
```

```
}

STATE initialising(){
  enable_motors();                        // enable motors
  Serial.println("INITIALISING");         // print the current stage
  return RUNNING;                         // return to RUNING STATE DIRECTLY
}

STATE running(){
  read_serial_command();                  // read command from serial communication

    speed_change_smooth();               //function to speed up and slow down smoothly

  return RUNNING;                         // return to RUNNING STATE again, it will run the RUNNING
}                                         // STATE REPEATLY

STATE stopped(){
disable_motors();                        // disable the motors
}


void speed_change_smooth()               // change speed, called in RUNING STATE
{
  speed_val += speed_change;    // speed value add on speed change
   if(speed_val > 500)    // make sure the  speed value less than the maximum
   speed_val = 500;
   speed_change = 0;    //make speed change equals 0 after updating the speed value
}

void read_serial_command()                // this function is called in RUNING STATE
{
  if (Serial.available()){
```

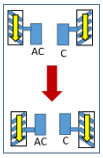
```
char val = Serial.read();              // get the input from serial communication
Serial.print("Speed:");                //   print the current speed
Serial.print(speed_val);
Serial.print(" ms ");

switch(val){
  case 'w':
  case 'W':
   forward ();                          // call function forward if the typed command is “w” or “W”
   Serial.println("Forward");
   break;
  case 's':
  case 'S':
   reverse ();                          // call function reverse if the command is “s” or “S”
   Serial.println("Backward");
   break;
  case 'q':
  case 'Q':
   strafe_left ();                      // call function strafe_left if the command is “q” or “Q”
   Serial.println("Strafe Left");
   break;
  case 'e':
  case 'E':
   strafe_right ();                     // call function strafe_right if the command is “e” or “E”
   Serial.println("Strafe Rightt");
   break;
  case 'a':
  case 'A':
   ccw ();                              // call function ccw if the command is “a” or “A”
   Serial.println("ccw");
   break;
  case 'd':
```

```
      case 'D':
        cw ();                                  // call function cw if the command is “d” or “D”

        Serial.println("cw");
        break;
      case '-':
      case '_':
        speed_change = -100;  // make the speed change equals 100
        Serial.println("-100");
        break;
      case '=':
      case '+':
        speed_change = 100;             // make the speed change equals 100
        Serial.println("+");
        break;
       case 'p':
       case 'P':
      //default:
        stop ();
        Serial.println("stop");
        break;
    }
  }
}


void disable_motors(){                          // function to disable all motors, called in  STOPPED STATE
  left_front_motor.detach();
  left_rear_motor.detach();
  right_rear_motor.detach();
  right_front_motor.detach();
}
```

```
void enable_motors() {                    //enable all motors, was called in INITIALZING SATE
  left_front_motor.attach(left_front);
  left_rear_motor.attach(left_rear);
  right_rear_motor.attach(right_rear);
  right_front_motor.attach(right_front);
}

void stop(){                              // stop motors
  left_front_motor.writeMicroseconds(1500);
  left_rear_motor.writeMicroseconds(1500);
  right_rear_motor.writeMicroseconds(1500);
  right_front_motor.writeMicroseconds(1500);
}
```

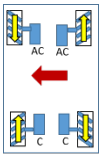



```
void forward(){                           // moving forward
  left_front_motor.writeMicroseconds(1500 + speed_val);
  left_rear_motor.writeMicroseconds(1500 + speed_val);
  right_rear_motor.writeMicroseconds(1500 - speed_val);
  right_front_motor.writeMicroseconds(1500 - speed_val);
}
```

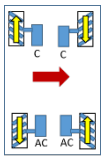



```
void reverse(){                           // reverse
  left_front_motor.writeMicroseconds(1500 - speed_val);
  left_rear_motor.writeMicroseconds(1500 - speed_val);
  right_rear_motor.writeMicroseconds(1500 + speed_val);
  right_front_motor.writeMicroseconds(1500 + speed_val);
}
```

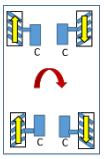
```
void strafe_left(){                                    // straight left
  left_front_motor.writeMicroseconds(1500 - speed_val);
  left_rear_motor.writeMicroseconds(1500 + speed_val);
  right_rear_motor.writeMicroseconds(1500 + speed_val);
  right_front_motor.writeMicroseconds(1500 - speed_val);
}
```



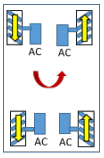

```
void strafe_right(){                                   //straight right
  left_front_motor.writeMicroseconds(1500 + speed_val);
  left_rear_motor.writeMicroseconds(1500 - speed_val);
  right_rear_motor.writeMicroseconds(1500 - speed_val);
  right_front_motor.writeMicroseconds(1500 + speed_val);
}
```

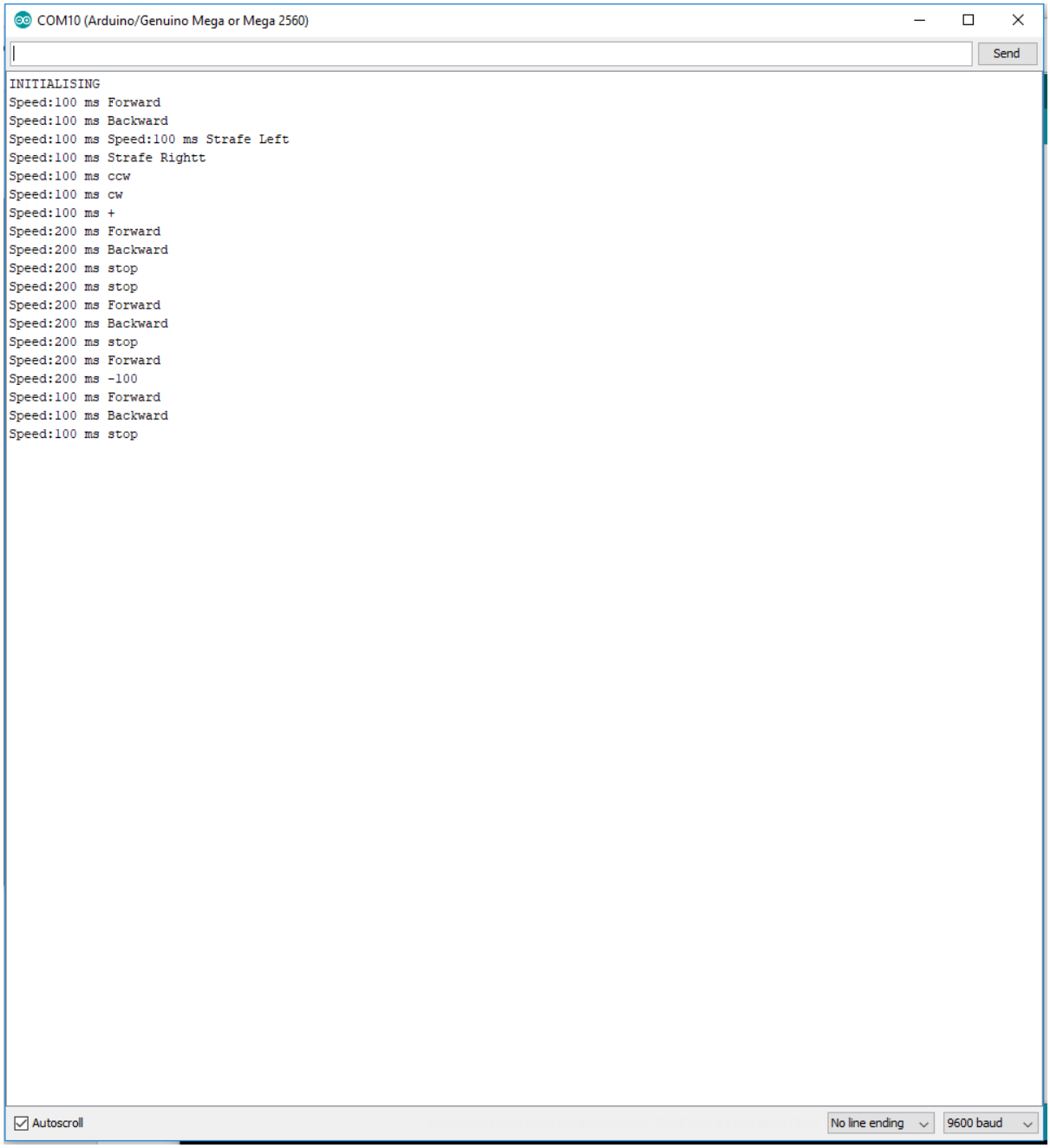



```
void cw(){                                             //clockwise
  left_front_motor.writeMicroseconds(1500 + speed_val);
  left_rear_motor.writeMicroseconds(1500 + speed_val);
  right_rear_motor.writeMicroseconds(1500 + speed_val);
  right_front_motor.writeMicroseconds(1500 + speed_val);
}
```




```
void ccw(){                                            //anticlockwise
  left_front_motor.writeMicroseconds(1500 - speed_val);
  left_rear_motor.writeMicroseconds(1500 - speed_val);
  right_rear_motor.writeMicroseconds(1500 - speed_val);
  right_front_motor.writeMicroseconds(1500 - speed_val);
}]
```




COM10 (Arduino/Genuino Mega or Mega 2560)
Send
INITIALISING
Speed:100 ms Forward
Speed:100 ms Backward
Speed:100 ms Speed:100 ms Strafe Left
Speed:100 ms Strafe Rightt
Speed:100 ms ccw
Speed:100 ms cw
Speed:100 ms +
Speed:200 ms Forward
Speed:200 ms Backward
Speed:200 ms stop
Speed:200 ms stop
Speed:200 ms Forward
Speed:200 ms Backward
Speed:200 ms stop
Speed:200 ms Forward
Speed:200 ms -100
Speed:100 ms Forward
Speed:100 ms Backward
Speed:100 ms stop
Autoscroll
No line ending
9600 baud

Distance Measuring Infrared Sensors

Infrared sensor interface, and Arduino programming for measurement of object distance (What is the sensor/specs? What does it do? How is the sensor connected to Arduino, what simple codes are needed to get in the signals, how are they processed?)

Distance measuring infrared sensor information

The infrared we use is to measure the distance between the robot and other objects. The infrared proximity sensor measures a distance by omitting a beam of infrared light and used a position-sensible photo detector (PSD) to determine the distance using triangulation principle for distance measurement. The effective distance measuring range for the proximity sensor is varied for different versions.

According to the datasheet of the sensor, we can see that the sensor has IR LED equipped with lens and a position-sensible detector (PSD). And the circuit is designed on triangulation principle for distance measurement. Infrared LED can emits narrow light beam. After reflating from the object, the beam will be directed through the second lens on a position-sensible detector. The conductivity of the PSD is related to the position where the beam falls. The conductivity is converted to voltage, and the distance can be calculated. The output of the distance sensor is inversely proportional, this means that when the distance is growing the output is decreasing. All sensors have their specific measuring range and the range depends on the type of the sensor.

Operating voltage: 4.5V -5.5V ;

Average current consumption: 30mA;

Output type: analog voltage 0~3.5V

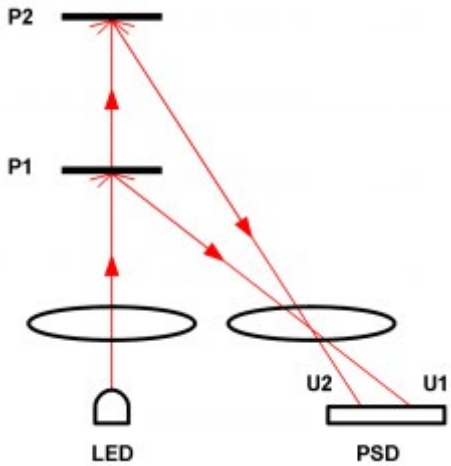


Fig. 14 Working principle of IR Distance sensor

https://wiki.eprolabs.com/index.php?title=IR_Distance_Sensor_GP2YOA21YK

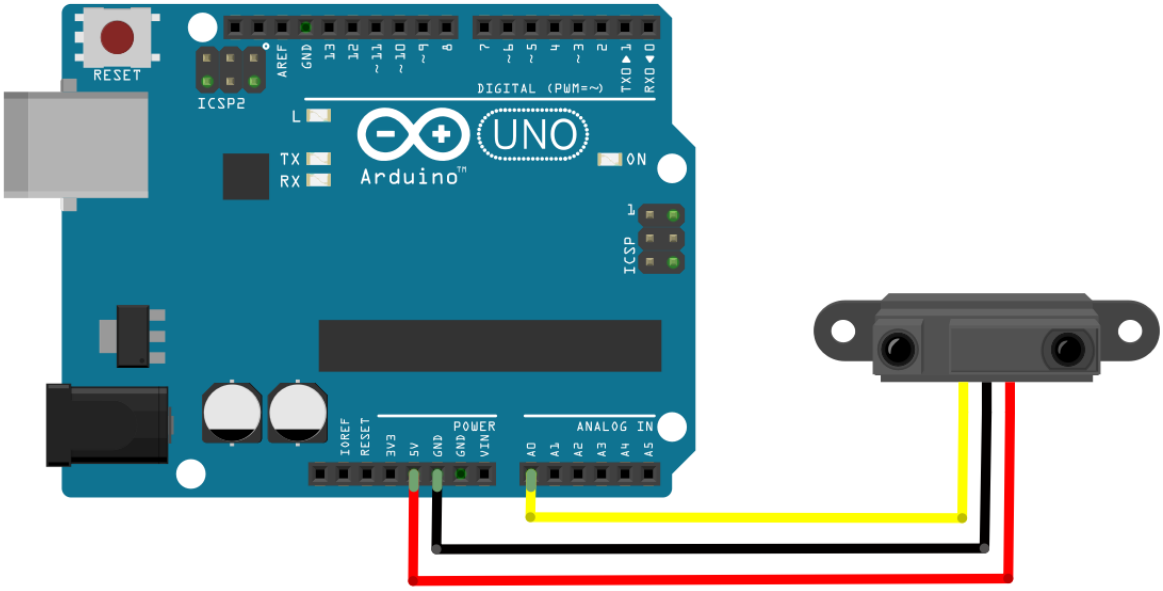
Hardware connection with Arduino

The sensor has 3 pins wires which connects it to the Arduino.

The red wire goes to 5V

The black goes to GND

Yellow wire goes to A0 (Analog pin for Arduino)



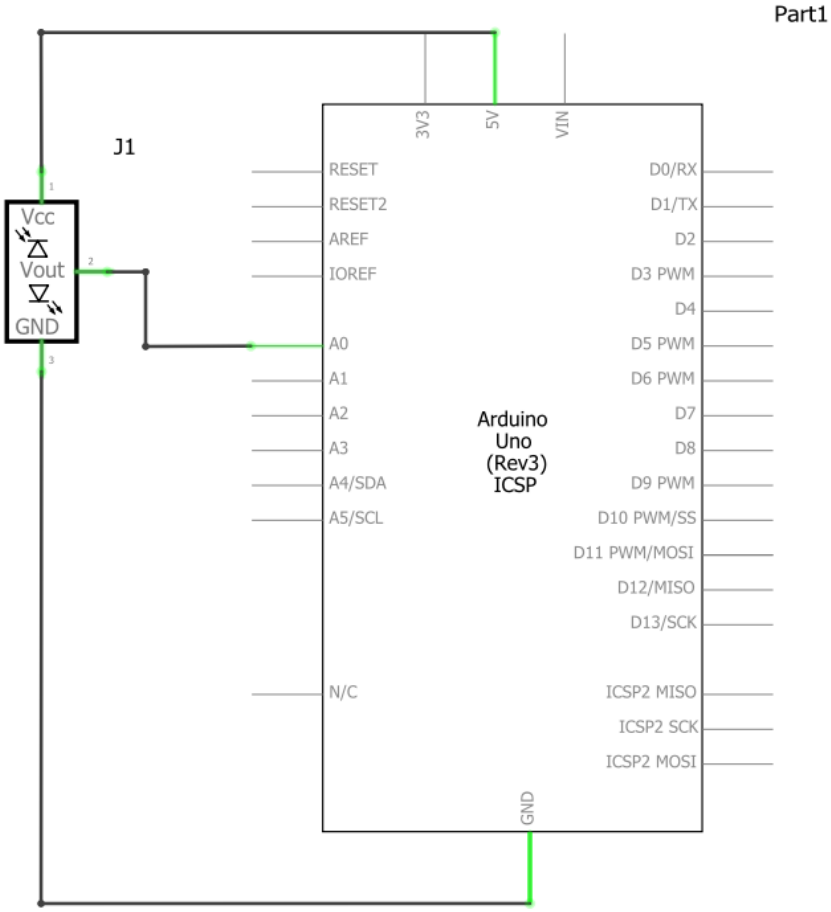


Fig.15 Infrared distance sensor hardware connection with Arduino

Sample codes

– for testing the sensor function and comparing the accuracy of calibrated data graph and data graph from data sheet.

On the datasheet of the sensor the graph of the ralation between output voltage and measured distance. We can note that the ralation is not linear one. In order to make our program more easy to write, we need to find a equation to descibe the voltage/distance relation. The other way to get more precise votage-distance relation is to calibrate the sensor by yourselt and draw the ralation graph and then using cuve fitting to find a equation decribe the relation. You can using Excel to do the curve ftting process.

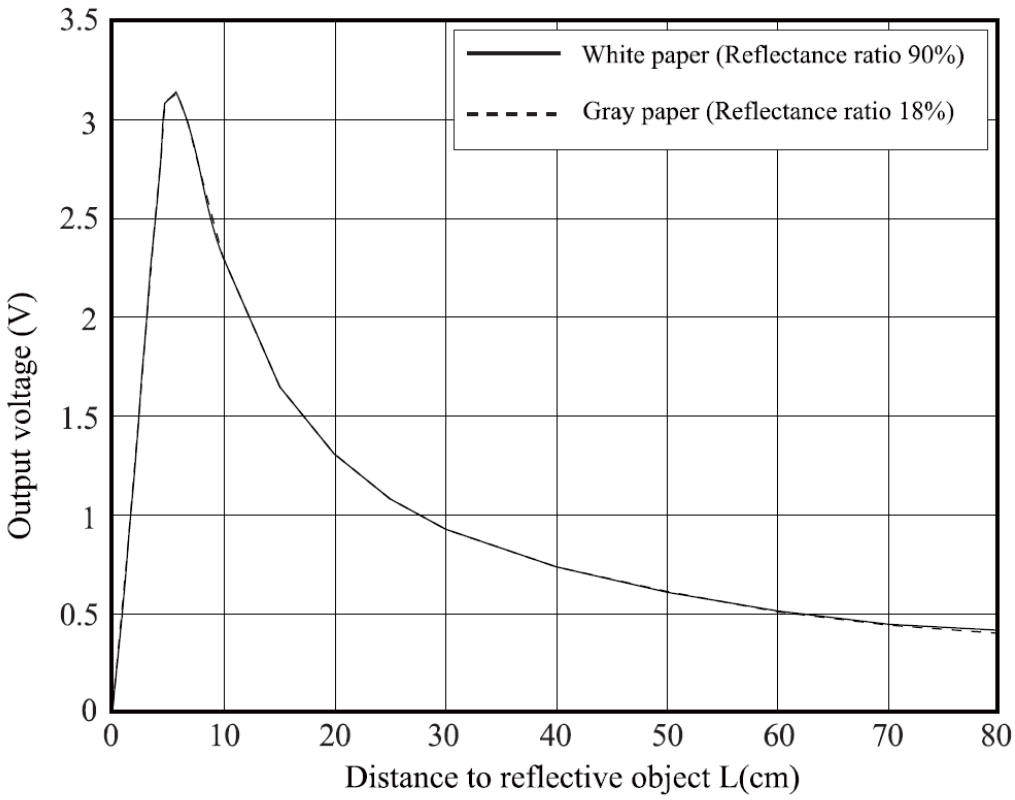
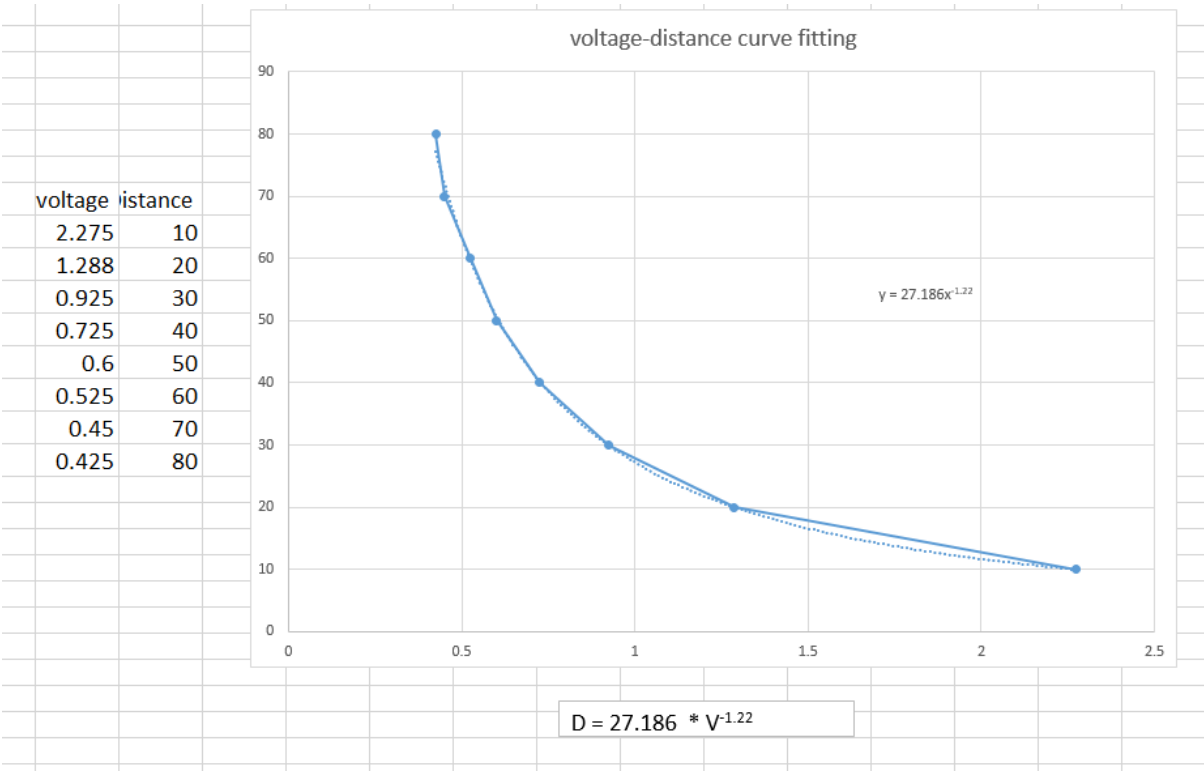
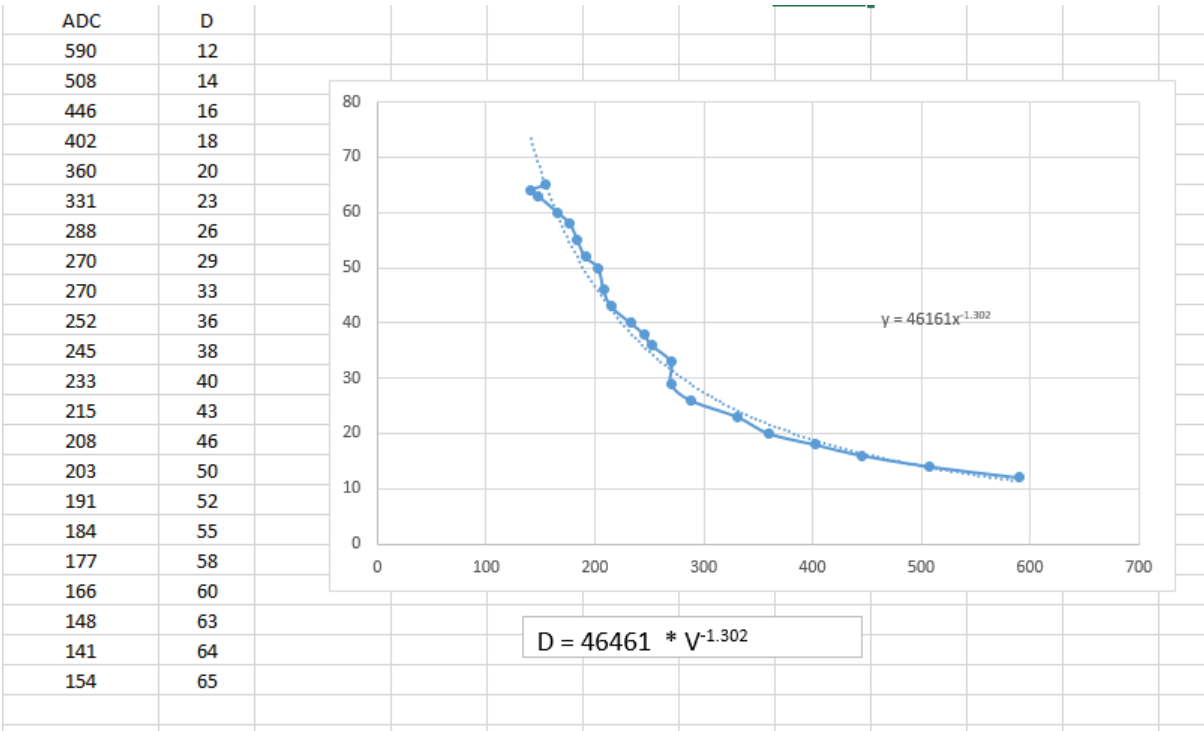


Fig.16 Voltage and distance relation of infrared sensor



voltage	istance
2.275	10
1.288	20
0.925	30
0.725	40
0.6	50
0.525	60
0.45	70
0.425	80

Fig.17 Data sheet graph curve fitting



ADC	D
590	12
508	14
446	16
402	18
360	20
331	23
288	26
270	29
270	33
252	36
245	38
233	40
215	43
208	46
203	50
191	52
184	55
177	58
166	60
148	63
141	64
154	65

Fig.18 Calibrated data graph curving fitting

Codes

```
int irsensor = A0;     //sensor is attached on pinA0

byte serialRead = 0;  //for control serial communication
```

```
int signalADC = 0;  // the read out signal in 0-1023 corresponding to 0-5v

void setup() {
  // put your setup code here, to run once:
Serial.begin(9600);    // start serial communication
}

void loop() {
  // put your main code here, to run repeatedly:

      if (Serial.available())           // Check for input from terminal
   {
     serialRead = Serial.read();     // Read input
     if (serialRead==49)           // Check for flag to execute, 49 is ascii for 1, stop serial printing
     {
       Serial.end();                 // end the serial communication to get the sensor data
     }
   }

  signalADC = analogRead(irsensor);   // the read out is a signal from 0-1023 corresponding to 0-5v
  int distance1 = 17948*pow(signalADC,-1.22);  // calculate the distance using the datasheet graph
  int distancec = 46161*pow(signalADC,-1.302);  // calculate the distance using the calibrated graph
Serial.print("distance datasheet graph  "); //print out the results on serial monitor
Serial.print(distance1);
Serial.println("cm");
Serial.print(" ");
Serial.print("distance calibration graph  ");
Serial.print(distancec);
Serial.println("cm");
}
```

COM5 (Arduino Due (Programming Port))

Send

```
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 12cm    distance calibration graph 19cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 12cm    distance calibration graph 19cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 25cm
distance datasheet graph 15cm    distance calibration graph 25cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 12cm    distance calibration graph 19cm
distance datasheet graph 15cm    distance calibration graph 25cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 25cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 12cm    distance calibration graph 19cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
distance datasheet graph 15cm    distance calibration graph 24cm
```

Autoscroll

No line ending

9600 baud

Gyroscope Sensor (BNO08X)

Gyroscope sensor interface, and Arduino programming for measurement of the angular velocity (What is the sensor/specs? What does it do? How is the sensor connected to Arduino, what simple codes are needed to get in the signals, how are they processed?)

Gyroscope sensor information

Gyro sensing is a device that measures the angular velocity. Angular velocity is a term to describe the change in rotation angle per unit of time, so gyro is also known as angular ate sensor or angular velocity sensor. Most of MEMS Gyro sensor sense angular velocity are manufactured based on the Coriolis effects.

Coriolis force: $F_C = -2\mathrm{M}\overline{\omega} \times \overline{v}$. Form this equation we can know that the force is perpendicular to vector $\overline{\omega}$ and vector $\overline{v}$. Assuming we have an object (mass M) having a velocity $\overline{v}$, and when an angular velocity $\overline{\omega}$ is applied to this object, the object will experience a Coriolis force in the direction of the arrow. We can see that the Coriolis force is proportional to the angular velocity $\overline{\omega}$.

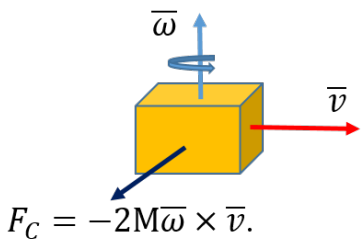
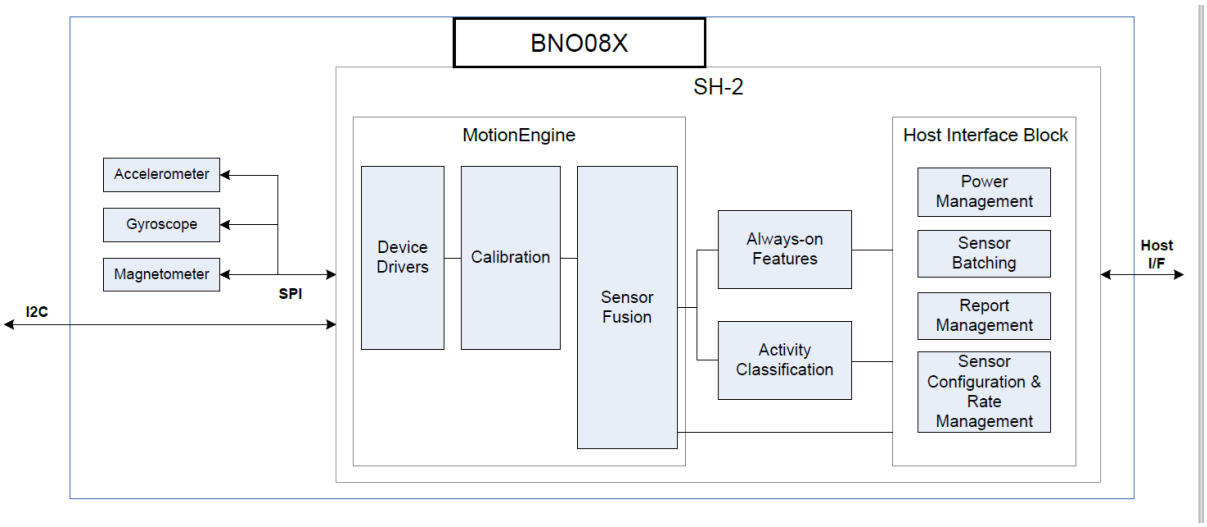


Fig.19 Coriolis effects

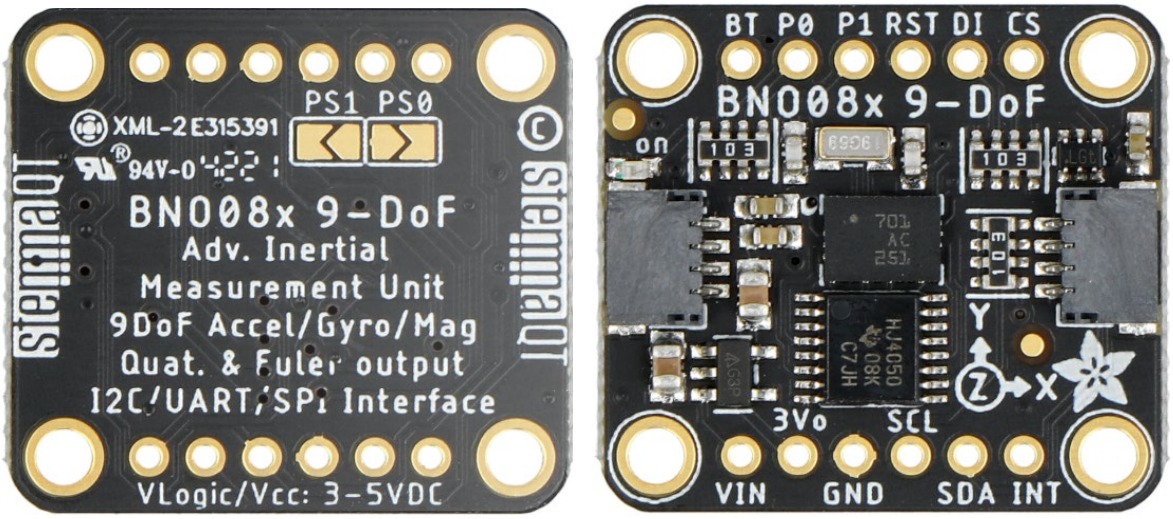
By designing different structure to change the Coriolis effects into electronic signals to measure the angular velocity. Take the tuning fork configuration gyro as an example, it converts the Coriolis force into capacity change.

Gyroscopes do not have a fixed reference, and only measure changes. The maximum angular rate of change (degree per seconds) is used to choose the proper gyroscope we need.

The gyroscope used in this project is integrated inside the BNO08X, which is a 9 DOF motion sensor including a triaxial gyroscope. In this work, only the Z-axis angular velocity is used. More about BNO08X refers to its data sheet placed in the Canvas/Files/Data Sheets.



(a) Block diagram



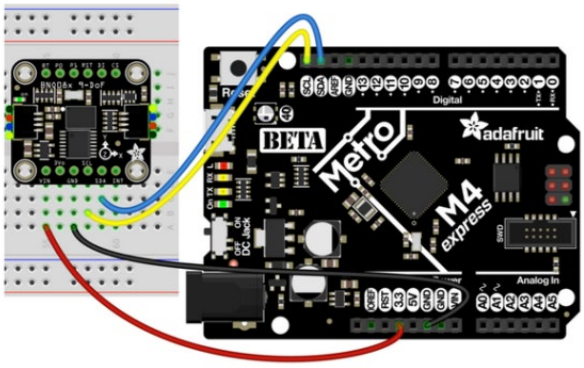
(b)Pin layout

Fig.20 BNO08X - A 9DOF Motion Sensor

Hardware connection with Arduino

The sensor communicates with the microcontroller through the I²C interface and provides uncalibrated angular velocity output in rad/sec. The uncalibrated output includes bias (no compensation applied). A positive value corresponds to counter-clockwise rotation about the Z-axis according to the right-hand rule. The specific information about this sensor can be found in its datasheet.

Note that the BNO board uses only <u>3.3V supply</u>, not 5V. It needs to be wired correctly to a microcontroller to avoid damage, referring to the images in Fig. 21.



- Connect board VIN (red wire) to Board 3V
- Connect board GND (black wire) to Board GND
- Connect board SCL (yellow wire) to Board SCL
- Connect board SDA (blue wire) to Board SDA

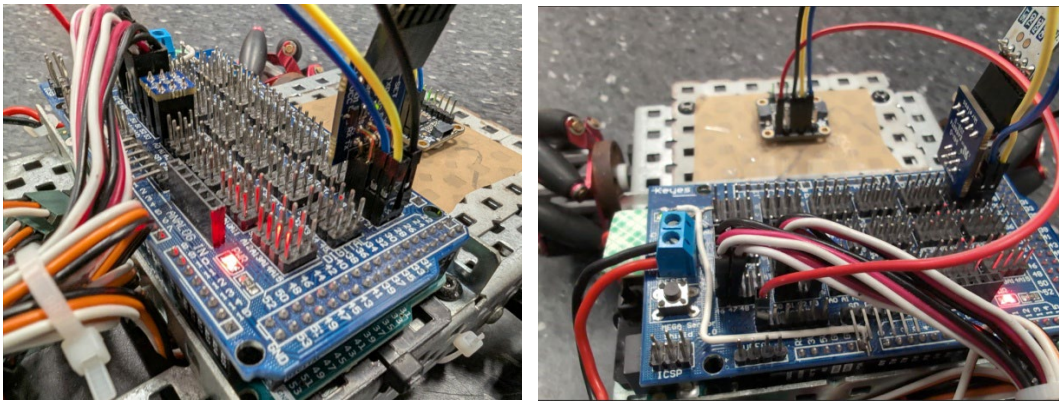


Fig.21 BNO08X Wired to Arduino/Microcontroller

Sample Codes – for testing the function of the gyro sensor

The gyroscope returns the current angular velocity in rad/s along the Z-axis and prints it to the serial monitor. The code is commented in RobotBaseCodes.ino for clarity.

Fig. 22 (blank)

Phototransistor

Fire sensor interface and Arduino programming for detection of a fire (What is the sensor/circuit? What does it do? How is the sensor connected to Arduino, what simple codes are needed to get in the signals, how are they processed?)

Phototransistor information

A phototransistor is a sensor which electrical resistance is altered depending on the light intensity falling on it. The fact is that the light striking the semiconductor frees electronics/holes and causes current flow in the based region. The more intensive is the light, the more free carrier are formed and the lower the resistance of the element. Therefore, it is a device to convert the light energy to an electrical signal, when the phototransistor exposed in the light.

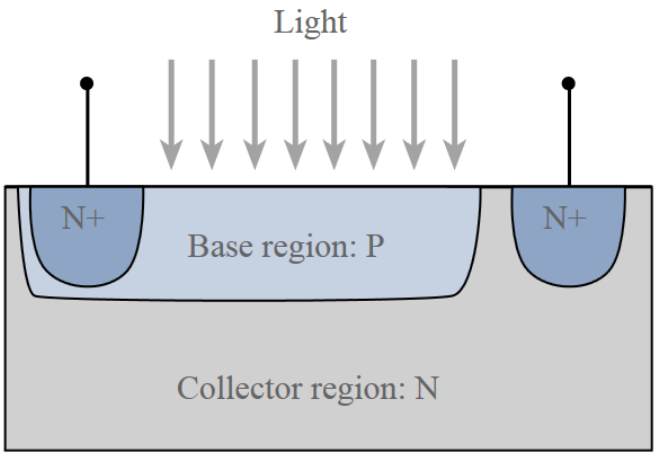


Fig.23 Phototransistor Structure

https://www.electronics-notes.com/articles/electronic_components/transistor/what-is-a-phototransistor-tutorial.php

The phototransistor is similar to the regular Bipolar Junction Transistor. It uses the basic transistor concept as the basis of its operation, but the base current is produced and controlled by light instead of voltage. It is able to sense the light level and alter the current flowing between emitter and collector according to the level of the light it receives.

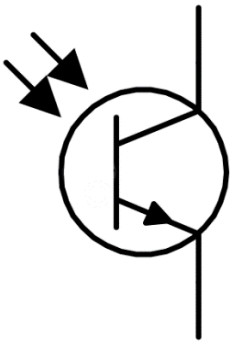


Fig.24 Electrical symbol of phototransistor

We can design a simple circuit to read out the signal controlled by phototransistor. We can use Arduino to read out the voltage of V_{out}. When the light is striking on the phototransistor, the voltage drop across the device decreases and more current flow through it. The resistor between read out pin and ground

creates a second voltage drop that is proportional to the amount of current flowing through the phototransistor. An intensive light creates a higher voltage out.

The V_{out} change is because the phototransistor let more current pass when more lights shining on it, or less current with less light. When the current flow through the resistor, the more current pass through the resistor, the voltage would be higher. When less current pass, the voltage will be lower.

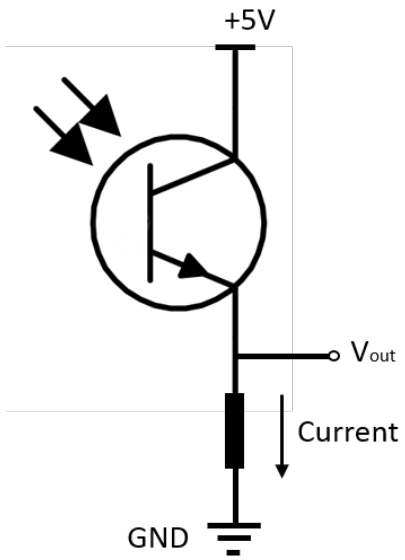


Fig.25 Simple circuits for phototransistor **(in active mode)**

Knowing this will help us choose the value of the resistor. The higher value the resistance is, V_{out} will see higher voltage value for the same current. A 2kΩ resistor will give V_{out} higher voltage comparing to 1kΩ.

So, a smaller resistor in series with the phototransistor makes the circuit less sensitive to light, while a larger resistor makes the circuit more sensitive to light.

$$V_{out} = I \times R$$

Under the same level light, a 10kΩ resistor will give 5 times voltage value than 2kΩ resistor in series with the phototransistor.

Hardware connection with Arduino

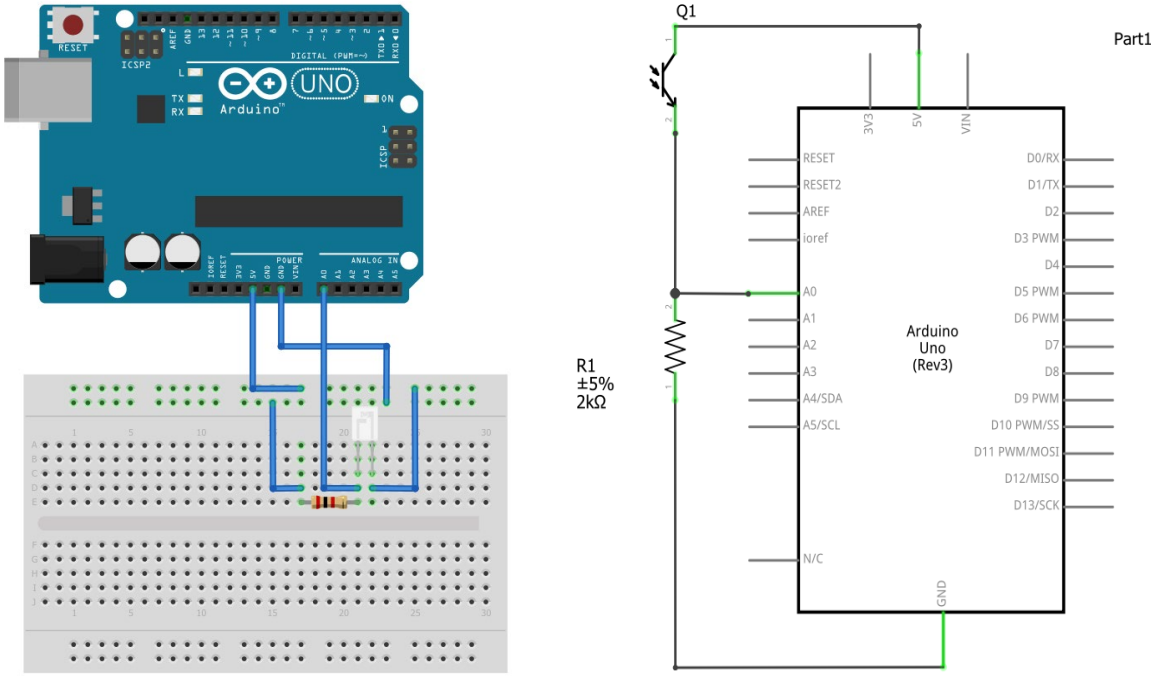
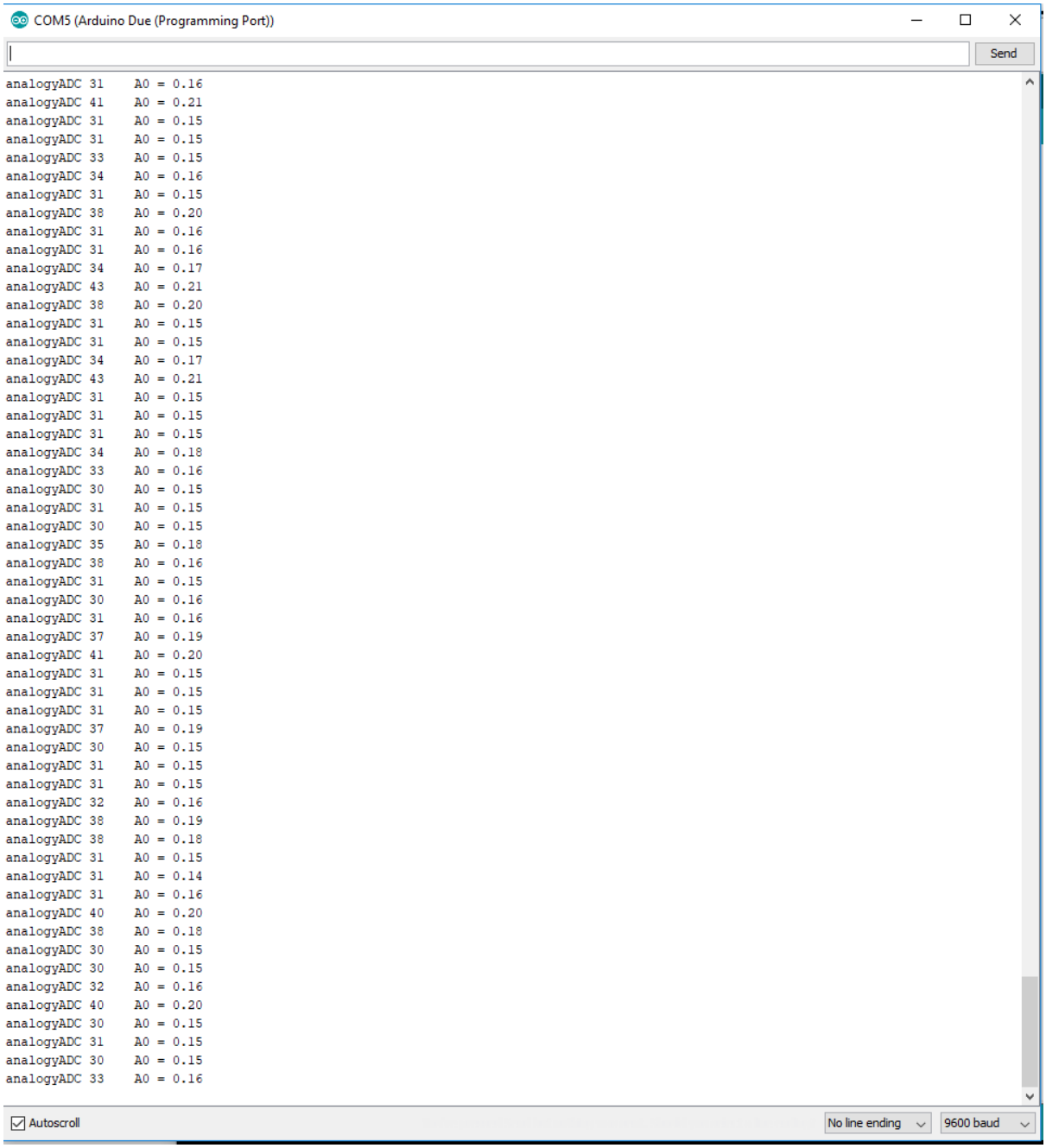


Fig.26 phototransistor hardware connection with Arduino

Sample Codes

– for testing the basic function of the phototransistor

```
int sensorValue = A0;

void setup() {
  // put your setup code here, to run once:
Serial.begin(9600);
}

void loop() {
  // put your main code here, to run repeatedly:

float volts = analogRead(sensorValue)*5.0/1024.0;
Serial.print("analogyADC ");
Serial.print(analogRead(sensorValue));
Serial.print("    ");
Serial.print("A0 = ");
```

```
Serial.println(volts);
delay(100);
}
```